Assignment 2

Topics

- Revision of WRAV201
- Stacks

Task 1: Array Implementation of Stack

Stacks are linear data structures that follow the Last-In-First-Out (LIFO) principle. They are commonly used in parsing, recursion, and memory management. In class you were shown a linked-list implementation. For this task you will be required to provide an alternate implementation using an array.

The array implementation should have the same class and public method signatures as the linked list version discussed in class. Other than the default constructor, which creates an array with an initial capacity of 10, there should also be a constructor that accepts the initial capacity of the stack as a parameter and creates the underlying array of this size.

Unlike the linked list implementation, it is possible for a stack to become too large for the underlying array to contain all the values when a push is called. To cater for this, when attempting to push a new value, if the stack is full, then the underlying array should be resized to double the current capacity (i.e. from 10 to 20, or 20 to 40, etc.) This resizing approach is commonly used in many programming languages (such as Java, C# and Python) and data structures (such as ArrayList) and is known as *dynamic resizing*.

Demonstrate that everything is working correctly by using the class examples (palindrome and matching brackets) and ensuring that they run correctly.

Task 2: Reverse Polish Notation (RPN) Calculator

Reverse Polish Notation (RPN), also known as *postfix* notation, is a mathematical notation in which every operator *follows* all its operands (instead of being *in*-between them). It eliminates the need for parentheses to define operation precedence. Expressions in regular mathematical notation are known as *infix* expressions.

Infix Expression	RPN Expression	Expected Result
2 + 3	2 3 +	5.0
2 * 3	2 3 *	6.0
1 + 2 + 3	1 2 + 3 +	6.0
1 + (2 + 3)	1 2 3 + +	6.0
1 + 2 * 3	1 2 3 * +	7.0
(1 + 2) * 3	1 2 + 3 *	9.0
5 + (1 + 2) * 4 - 3	5 1 2 + 4 * + 3 -	14.0
10 / 2	10 2 /	5.0
3 – (4 * 5)	3 4 5 * -	-17.0
(6 / 2) + 3	6 2 / 3 +	6.0

You are required to implement methods called `rpnStack` and `rpnRecursive` that evaluate a string containing a valid RPN expression and return the result. `rpnStack` is to use a stack to solve the problem, while `rpnRecursive` is to use a recursion to solve the problem.

Be sure to handle edge cases, e.g. division by zero (3 2 2 - /), not the correct number of operands (1 2 3 + + +), etc.

Demonstrate that your implementations work correctly by having them calculate the RPN expressions in the table above (at a minimum).

Task 3: HTML/XML Tag Validator

HTML[1] and XML[2] documents are text documents and use tags to define structure within the document. These tags must be properly nested and closed to ensure the document is valid. For example:

```
<div>
  <p>Hello <strong>world</strong></p>
  <hr/>
</div>
```

This is valid because every opening tag has a corresponding closing tag in the correct order[3].

All tags start with < and end with >. There are *three* types of tags:

- `<X>` is an **opening** tag with the name "X". This type of tag must have a matching closing tag.
- `</X>` is a **closing** tag with the name "X". This type of tag must have a matching opening tag.
- `<X/>` is self-closing tag with the name "X". This type of tag stands on its own and does *not* have a matching tag.

Input	Output	Notes
`<div><p></p></div>`	True	Valid nesting
`<div><p></div></p>`	False	Incorrect nesting
`<div><span></div>`	False	Missing
`<div><img src='x.jpg'/></div>`	True	Self-closing tag ignored

A tag has a **name**, followed by optional parameters in the format a='b' or a="b" (e.g. `<img src='x.jpg'/>` is a self-closing tag with the name `img` and has one parameter `src='x.jpg'`). Tag names are case-insensitive.

Write a method called `areTagsMatched` that accepts a String containing the XML and returns true if all the tags are matched correctly. If there are errors be sure to display what these are on the console (e.g. "Missing ").

---

[1] HTML is a *type of* XML document.
[2] XML documents are not only used for webpages, but the format is also widely used in a number of domains. Search to find out more, if interested. You will be seeing XML in WRPV301 next year again.
[3] Hopefully you see the similarity to the class example…

Write a method called `isMatchedXMLFile` that accepts a filename, and returns true or false depending on whether the contents of the file have matching tags or not.

Demonstrate that the methods are working correctly.

Rubric

Below is the rubric, available on Moodle, that will be used when assessing this Assignment.

Criteria	Marks	Option
Task 1: Array Implementation of Stack An array implementation of the Stack was to be provided: 1. Internally, the stack is represented as an array; 2. It has a default constructor which sets the capacity of the array to 10; 3. It has a constructor that accepts an initial capacity and sets the array capacity to this value; 4. Has implementations for peek, push, pop, isEmpty, clear and toString; 5. The push method resizes the internal array if the array is full when attempting to push a new value. 6. The class examples were to be used to demonstrate that the array implementation worked correctly.	0	Not done or does not compile.
	1	Some of the required functionality implemented.
	2	All the functionality implemented and works correctly.
Task 2: Reverse Polish Notation (RPN) Calculator (Stack Solution) A method called rpnStack was to be implemented that accepted a valid RPN expression and returned the result. The solution MUST use a stack to perform the calculation. Correctness was to be demonstrated by showing the outputs of the expressions in the table (at a minimum).	0	Not done or does not compile.
	1	Some of the required functionality implemented.
	2	All the functionality implemented and works correctly.
Task 2: Reverse Polish Notation (RPN) Calculator (Recursive Solution) A method called rpnRecursive was to be implemented that accepted a valid RPN expression and returned the result. The solution MUST be a recursive solution and should NOT use a stack. Correctness was to be demonstrated by showing the outputs of the expressions in the table (at a minimum).	0	Not done or does not compile.
	1	Some of the required functionality implemented.
	2	All the functionality implemented and works correctly.
Task 3: HTML/XML Tag Validator HTML/XML documents have matched tags that provide structure to a text document. The following was to be done: 1. A method called areTagsMatched was to be written. This method accepted a String and returned true if it had matched tags, false otherwise. If there were errors, these were to be displayed on the console; 2. A method called isMatchedXMLFile was to be written. This method accepted a filename and returned true if the contents of the HTML/XML file had matching tags, false otherwise;	0	Not done or does not compile.
	1	Some of the required functionality implemented.
	2	All the functionality implemented and works correctly.

3. The correctness of the methods was required. This would involve showing that each method produced the expected output for several cases that passed AND failed the checks.		
Code Presentation How presentable and understandable was the code? Specifically: • Was the correct naming conventions used for Java? i.e. camel-case for identifiers, with classes being capitalised, method and field identifiers not. • Were appropriate identifiers used for variables, fields, parameters, methods, etc? Did they make sense? Were they unambiguous? • Was the code properly formatted? i.e. was proper indentation used, single statement per line? Etc. • Was the code commented where needed? i.e. was it explained what the code was doing?	0	Difficult to read code. Very poor.
	1	Some of the requirements were met, but not all.
	2	All the requirements mentioned were applied.